ENGINEER TO EXCEL

PRINTER QUEUE MANAGEMENT USING QUEUE DATA STRUCTURE

CSA0306- DATA STRUCTURES

Supervised By:
Dr. Shakila M.

Presented By:
Lochana Sri R.S. (192521348)

THE SHARED PRINTING PROBLEM

- **The Shared Resource Challenge**

  In standard network configurations (offices, computer labs), multiple users frequently transmit print instructions to a single shared physical printer concurrently.

- **Inherent Hardware Bottleneck**

  A physical printer is fundamentally sequential; it cannot print more than one document at any single millisecond, necessitating a structured scheduling mechanism.

- **The Need for Buffer Management**

  We require an asynchronous buffer system to securely host received documents and release them systematically when the mechanical components are idle.

QUEUE DATA STRUCTURE: FIFO

Enqueue()

Inserts a new print job into the queue at the rear index. Represents sending a new document to print.

Dequeue()

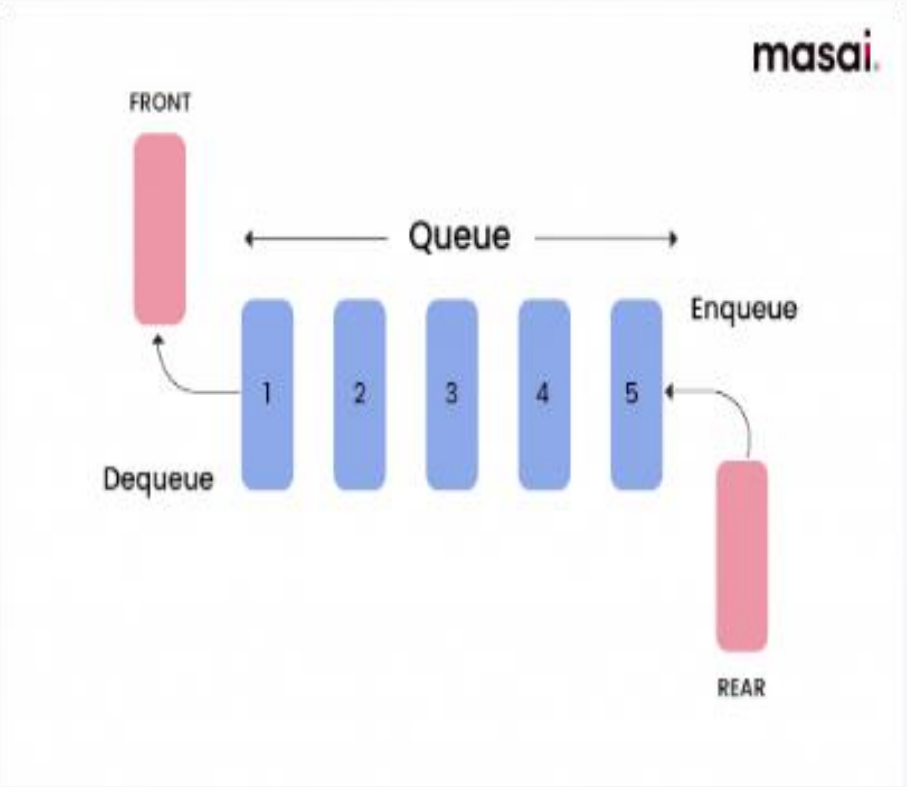
Removes the front job from the queue and prints it. Follows successful physical document release.

Peek() / Front()

Inspects the details (such as owner and size) of the document currently up next, without removing it from the queue.

IsEmpty()

Checks if there are any remaining print requests. Restores printer to energy-saving standby if empty.



STEP-BY-STEP PRINT QUEUE WORKING

1

Idle State

The printer is idle with no active print requests.

Front = -1
Rear = -1
Queue: [EMPTY]

2

Enqueue Jobs

Users send Job A, Job B, and Job C sequentially.

Front = 0 [Job A]
Rear = 2 [Job C]
Queue: [A, B, C]

3

Dequeue (Print)

Printer finishes processing Job A, dequeuing it.

Front = 1 [Job B]
Rear = 2 [Job C]
Queue: [B, C]

4

Dynamic Enqueue

New request Job D joins back of active queue.

Front = 1 [Job B]
Rear = 3 [Job D]
Queue: [B, C, D]

WHY QUEUE IS THE BEST CHOICE

Guaranteed Fairness

Adhering strict FIFO guarantees no user document gets starved or blocked by newer incoming requests.

Resource Decoupling

Frees the host computer immediately. The computer sends bytes in milliseconds, leaving the queue to feed the printer.

Optimal Overhead

Standard circular queue implementations execute boundary transitions without shifting elements, maintaining flat overheads.

Operation Time Complexities

Queue Operation	Time Complexity	Reason
Enqueue()	O(1)	Direct pointer insertion at Rear
Dequeue()	O(1)	Direct pointer extraction at Front
Peek()	O(1)	Instant read of Front index
Space Complexity	O(N)	Linear scale based on active jobs

COMPARISON WITH OTHER DATA STRUCTURES:

Data Structure	Ordering Principle	Insertion Cost	Deletion Cost	Printer Suitability	Practical System Consequence
Queue (FIFO)	**First In, First Out**	**O(1)**	**O(1)**	**Perfect**	**Excellent fairness, completely predictable execution.**
Stack (LIFO)	Last In, First Out	O(1)	O(1)	Poor	First user's job prints last. Causes severe starvation.
Array (Fixed-Size)	Index-based linear	O(1)	O(N)	Poor	Deleting element from front requires shifting all items back.
Linked List	Dynamic sequential	O(1)	O(1)	Moderate	Extremely dynamic but introduces larger memory pointer overhead.

CONCLUSION & WRAP UP

- Simplifies Shared Resource Management
- Ensures Deterministic O(1) Time Operations
- Provides Foundation for Complex Thread Schedulers
- Crucial Topic in Academic OS Architecture Modules

Thank You!